

# Краткий отчет по практическому заданию

**Студент:** Назарова Анастасия Денисовна

**Проект:** Консольный планировщик задач на C++

## Цель:

Разработать консольное приложение, позволяющее управлять личными задачами:

- Добавление задач с приоритетом и статусом
- Просмотр и фильтрация задач
- Удаление и изменение статуса
- Сортировка по приоритету

## Что реализовано:

1. Архитектура проекта разделена по классам:

- Task: структура задачи
- TaskManager: управление списком задач
- App: интерфейс пользователя (меню, обработка ввода)
- main.cpp: точка входа

2. Основной функционал:

- Добавление задачи
- Удаление задачи по ID
- Смена статуса задачи (выполнено / не выполнено)
- Фильтрация по статусу
- Сортировка задач по приоритету (по убыванию)
- Изменение приоритета задачи
- Проверка на пустые категории (например, “все задачи выполнены”)
- Печать задач после каждого действия, включая сортировку

3. Дополнительно:

- Список задач по умолчанию сортируется по ID
- После сортировки — автоматический вывод отсортированного списка
- Сообщения после удаления и смены статуса (Задача [id] удалена/выполнена)

## Сложности и решения:

| Проблема                                            | Решение                                                 |
|-----------------------------------------------------|---------------------------------------------------------|
| После сортировки по приоритету задачи не выводились | Добавила вызов printAllTasks() прямо в sortByPriority() |

| Проблема                             | Решение                                                                                                                              |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Сортировка: по ID или по приоритету? | Разделила логику: <code>printAllTasks()</code> — вывод по текущему порядку, <code>printAllTasksSortedById()</code> — отдельный вызов |

#### Технологии:

- Язык: C++17
- Среда: Xcode (Mac), совместимо с g++/clang++
- Хранение: в оперативной памяти (`std::vector<Task>`)

#### Результат

Рабочее приложение, полностью соответствующее ТЗ, со структурой, которая легко масштабируется (например, можно подключить графический интерфейс на SFML).